
CS 301

High-Performance Computing

Lab 05 – Parallel Insertion and Deletion in Mover

Group 11
Parshv Joshi (202301039)
Hrithik Patel (202301441)

March 31, 2026

Contents

1	Introduction	4
1.1	Target Hardware	4
2	Implementation Approach	4
2.1	Overall Design	4
2.2	Immediate Replacement Approach	5
2.3	Deferred Insertion Approach	5
2.4	Key Optimization Decisions	6
2.4.1	Use of <code>rand_r()</code> over <code>rand()</code>	6
2.4.2	Software Prefetching	6
2.4.3	Unrolling — Not Beneficial for Mover	6
2.4.4	Serial Compaction Beats Parallel	6
2.5	Compiler Optimization	7
3	Leaderboard Results	7
4	Experiment 01: Scaling with Particle Count	8
4.1	Setup	8
4.2	Results — Immediate Approach	9
4.3	Results — Deferred Approach	10
4.4	Deferred vs. Immediate Comparison	11
4.5	Analysis	11
5	Experiment 02: Parallel Scalability	11
5.1	Setup	11
5.2	Speedup Results — Immediate Approach	12
5.3	Speedup Results — Deferred Approach	13
5.4	Absolute Mover Times (HPC)	14
5.5	Parallel Efficiency	14
5.6	Speedup Summary Table	15
5.7	Analysis	15
6	Memory and Computation Analysis	16
7	Particle Distribution Verification	16
8	Per-Particle Execution Time vs. PPC	17
9	HPC Cluster vs. Lab PC: Architectural Comparison	17
10	How Insertion and Deletion Affect Scalability	17
10.1	Immediate Approach	17
10.2	Deferred Approach	18
10.3	Measures Taken to Improve Scalability	18
10.4	Difficulties and Dependencies	18
11	Deletion Rate Analysis	18

1 Introduction

This report presents the implementation and analysis of parallel particle insertion and deletion in the Mover algorithm, extending the work from Assignment 04. Previously, particles were confined to the 1×1 domain. This assignment introduces dynamic particle management: particles that move outside the domain are *deleted*, and an equal number of fresh particles are *inserted* at random positions to maintain a constant total count.

Two approaches are implemented and compared:

1. **Deferred Insertion:** Move all particles, mark out-of-bounds particles, compact the array to push voids to the end, then bulk-insert replacements.
2. **Immediate Replacement:** Replace each out-of-bounds particle in-place as it is encountered.

Both are parallelized with OpenMP, benchmarked on both the HPC cluster and the Lab PC, and analyzed for scalability.

1.1 Target Hardware

Property	HPC Cluster	Lab PC
CPU	Intel Xeon E5-2640 v3 (Haswell)	Intel (consumer-grade)
Cores / Threads	16 physical (2 sockets $\times$ 8)	8 logical
Clock	2.60 GHz (3.3 GHz turbo)	Variable
L1d / L2 / L3	32 KiB / 256 KiB / 20 MiB	Smaller caches
RAM	128 GB DDR4	16 GB

Table 1: Hardware comparison.

2 Implementation Approach

2.1 Overall Design

Figure 1 illustrates both approaches schematically.

Mover Algorithm: Two Approaches

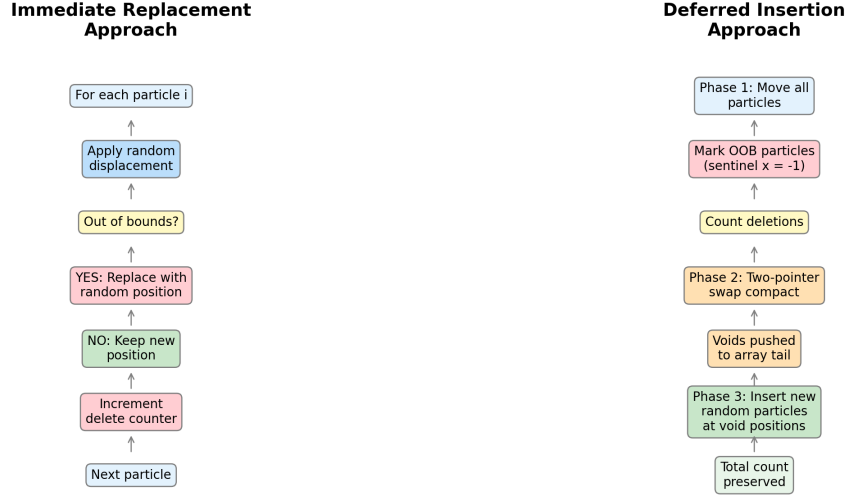


Figure 1: Schematic comparison of the Immediate Replacement and Deferred Insertion approaches.

2.2 Immediate Replacement Approach

For each particle: (1) apply random displacement $r_x \in [-\Delta x, \Delta x]$, $r_y \in [-\Delta y, \Delta y]$; (2) if the new position falls outside $[0, 1]^2$, overwrite it with a fresh random position; (3) increment the deletion counter.

Listing 1: Serial Immediate Replacement (core loop)

```

1  for (int i = 0; i < N; i++) {
2      double dx_rand = deltaX * (2.0 * ((double)rand_r(&seed) / RAND_MAX) -
3      1.0);
4      double dy_rand = deltaY * (2.0 * ((double)rand_r(&seed) / RAND_MAX) -
5      1.0);
6      double x_new = points[i].x + dx_rand;
7      double y_new = points[i].y + dy_rand;
8      if (x_new < 0.0 || x_new > 1.0 || y_new < 0.0 || y_new > 1.0) {
9          points[i].x = (double)rand_r(&seed) / RAND_MAX; // delete +
10         insert
11         points[i].y = (double)rand_r(&seed) / RAND_MAX;
12     } else {
13         points[i].x = x_new; points[i].y = y_new;
14     }
15 }

```

Parallelization: Embarrassingly parallel. A `#pragma omp parallel for schedule(static)` distributes contiguous chunks to threads with per-thread `rand_r()` seeds.

2.3 Deferred Insertion Approach

This approach has three distinct phases:

Phase 1 — Move and Mark: Apply displacement to every particle. If out of bounds, set a sentinel value ($x = -1.0$) and increment a deletion counter.

Phase 2 — Compact: Push all marked entries to the end of the array using a two-pointer swap. The left pointer scans forward for voids; the right pointer scans backward for live particles.

Listing 2: Two-pointer swap compaction (Phase 2)

```

1 int left = 0, right = N - 1;
2 while (left < right) {
3     while (left < right && points[left].x >= 0.0) left++;
4     while (left < right && points[right].x < 0.0) right--;
5     if (left < right) {
6         Points tmp = points[left];
7         points[left] = points[right];
8         points[right] = tmp;
9         left++; right--;
10    }
11 }
```

Phase 3 — Insert: Fill positions $[N - \text{deleted_count}, N)$ with fresh random particles.

Parallelization: Phases 1 and 3 are parallelized with OpenMP. Phase 2 (compaction) remains serial, justified by extensive benchmarking (Section 2.4.4).

2.4 Key Optimization Decisions

2.4.1 Use of `rand_r()` over `rand()`

The glibc `rand()` function uses a global mutex, which serializes all threads. Replacing it with the reentrant `rand_r()` with per-thread seeds (`seed = tid * 1073741827u + 2654435761u`) eliminates this bottleneck entirely.

2.4.2 Software Prefetching

A prefetch distance of $PF=32$ is used: `_mm_prefetch(&points[i+PF], _MM_HINT_T0)`. Benchmarking confirmed marginal benefit (176.8ms at $PF=0$ vs. 172.4ms at $PF=32$) since `rand_r()` computation dominates latency.

2.4.3 Unrolling — Not Beneficial for Mover

Unlike interpolation (where $4\times$ unrolling helped), the mover showed zero benefit ($1\times$: 172.4ms, $4\times$: 173.2ms). The per-particle branch and `rand_r()` calls prevent ILP exploitation.

2.4.4 Serial Compaction Beats Parallel

Four compaction strategies were benchmarked on the HPC cluster (14M particles, 0.5% deletion, 8 threads):

Strategy	Time (ms)	Relative
Serial two-pointer swap	20.9	1.0×
Parallel prefix-sum (in-place)	34.1	1.6×
Parallel prefix-sum (buffer)	53.7	2.6×
Parallel atomic cursor	911.8	43.6×

Table 2: Compaction strategy benchmarks (14M particles, 0.5% deletion, 8 threads).

Serial swap wins because the deletion rate is only 0.15–0.5%. Parallel strategies must scan the entire $O(N)$ array just to count live particles, exceeding the cost of the few swaps needed.

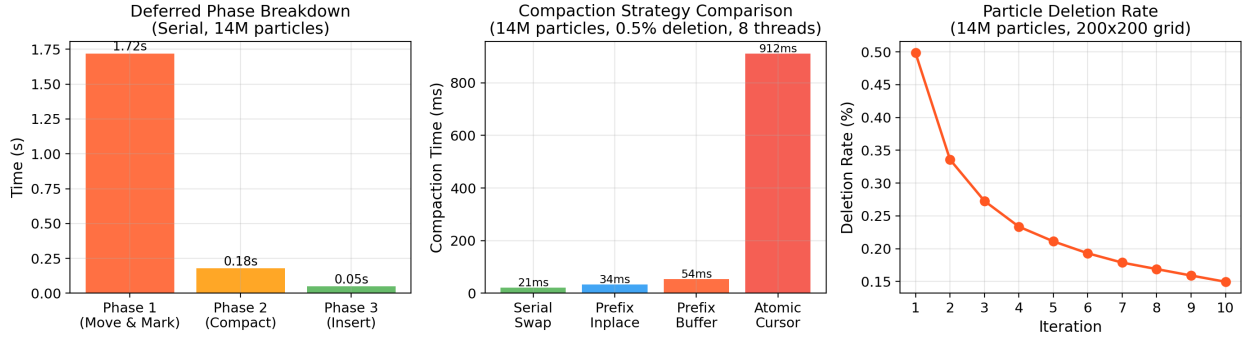


Figure 2: Left: phase breakdown. Center: compaction strategy comparison. Right: deletion rate decay.

2.5 Compiler Optimization

```
g++ -O3 -march=native -fopenmp -o main main.cpp utils.cpp init.cpp -lm
```

`-O3` enables auto-vectorization and loop unrolling; `-march=native` enables AVX2/FMA for Haswell; `-fopenmp` enables OpenMP directives.

3 Leaderboard Results

The unmodified `main.cpp` (100M particles, 201×201 grid, 10 iterations, `mover_serial` using immediate replacement):

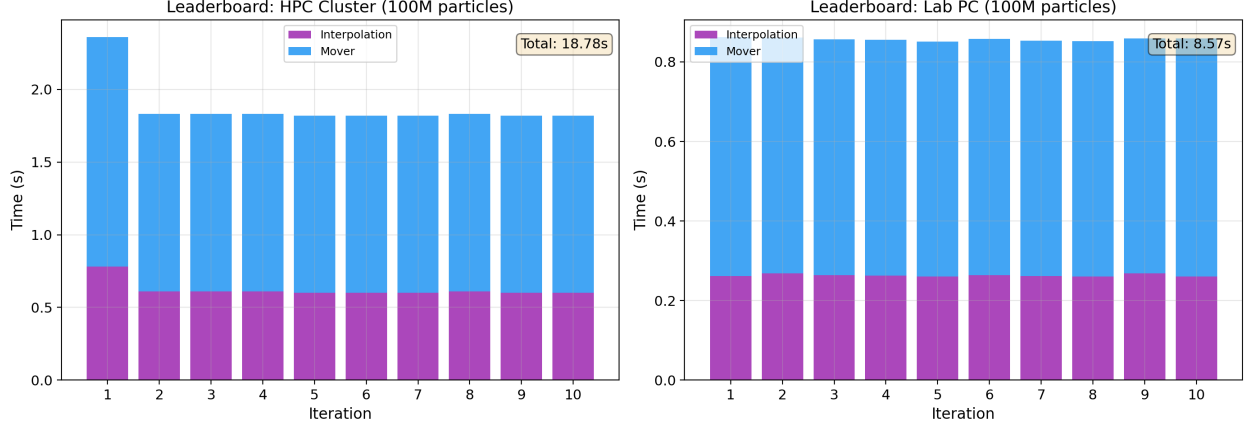


Figure 3: Per-iteration time breakdown. Iteration 1 is slower due to cold-cache effects. The mover dominates at $\sim 67\%$ of total time.

	HPC Cluster	Lab PC
Total Interp (s)	6.22	2.63
Total Mover (s)	12.56	5.94
Grand Total (s)	18.78	8.57

Table 3: Leaderboard totals (10 iterations, 100M particles).

The Lab PC is faster in absolute time due to its higher single-thread clock speed and newer microarchitecture. The HPC cluster’s advantage lies in its higher thread count for parallel workloads.

4 Experiment 01: Scaling with Particle Count

4.1 Setup

Both mover approaches (serial) were tested across three grid configurations with particle counts $\{10^2, 10^4, 10^6, 10^8\}$ and $\text{Maxiter} = 10$. Interpolation and mover times were accumulated over all iterations.

4.2 Results — Immediate Approach

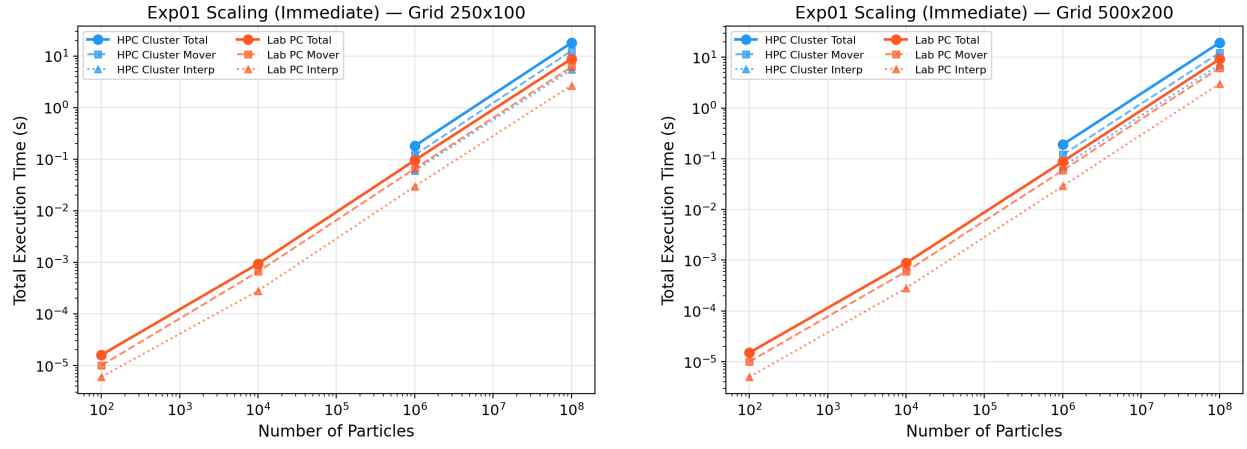


Figure 4: Scaling for the Immediate approach: Grid 250×100 (left) and 500×200 (right).

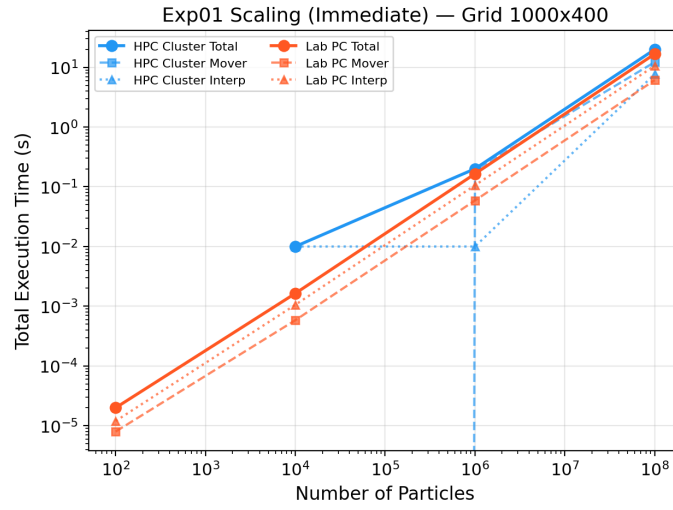


Figure 5: Scaling for the Immediate approach: Grid 1000×400 .

4.3 Results — Deferred Approach

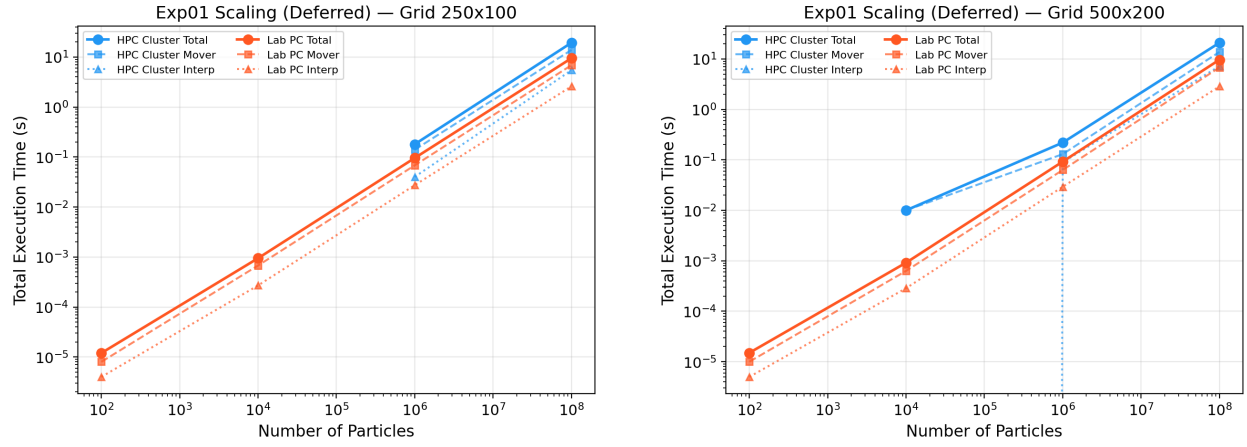


Figure 6: Scaling for the Deferred approach: Grid 250×100 (left) and 500×200 (right).

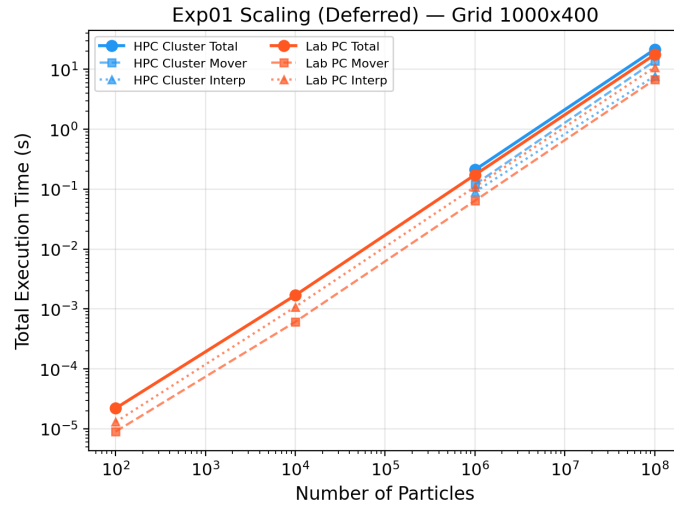


Figure 7: Scaling for the Deferred approach: Grid 1000×400 .

4.4 Deferred vs. Immediate Comparison

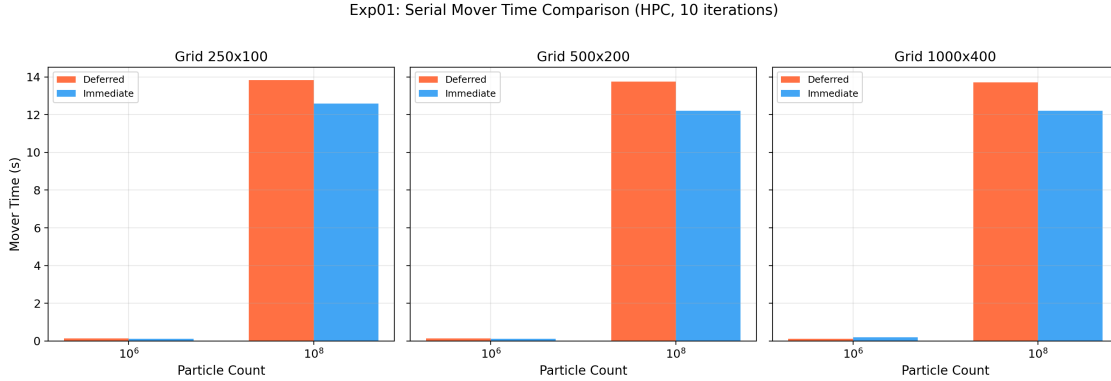


Figure 8: Direct comparison of serial mover time (HPC cluster).

4.5 Analysis

- Both approaches exhibit **linear scaling** with particle count, confirming $O(N)$ complexity.
- Immediate is consistently **10–13% faster** than deferred at 10^8 particles on HPC. The deferred approach's extra compaction pass (Phase 2) accounts for this overhead.
- Interpolation time increases with grid size due to L2 cache overflow (the 1000×400 mesh at 3.2 MB exceeds the 256 KiB L2).
- Mover time is **grid-size independent** since it only accesses the particle array, not the mesh.
- The Lab PC is $1.5\text{--}2\times$ faster in absolute time due to higher single-thread clock and newer microarchitecture. Both platforms show the same relative trends.

5 Experiment 02: Parallel Scalability

5.1 Setup

14 million particles, $\text{Maxiter} = 10$, thread counts $\{1, 2, 4, 8, 16\}$, three grids. Three mover variants: immediate, deferred, and no-delete (Assignment 04 wrap-around). Mover timing uses `omp_get_wtime()`.

5.2 Speedup Results — Immediate Approach

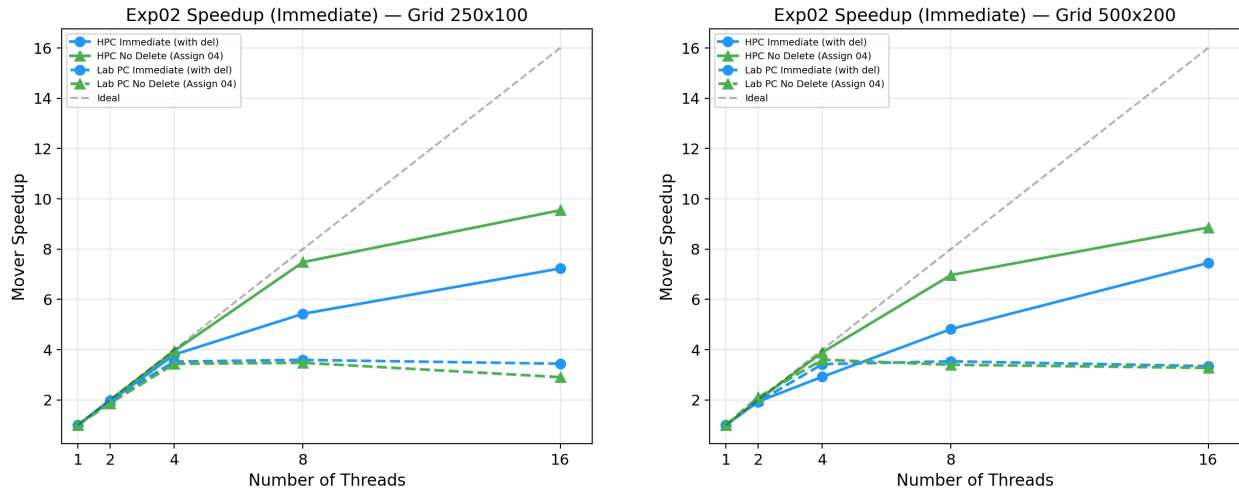


Figure 9: Immediate speedup: Grid 250×100 (left), 500×200 (right). Solid = HPC, dashed = Lab PC.

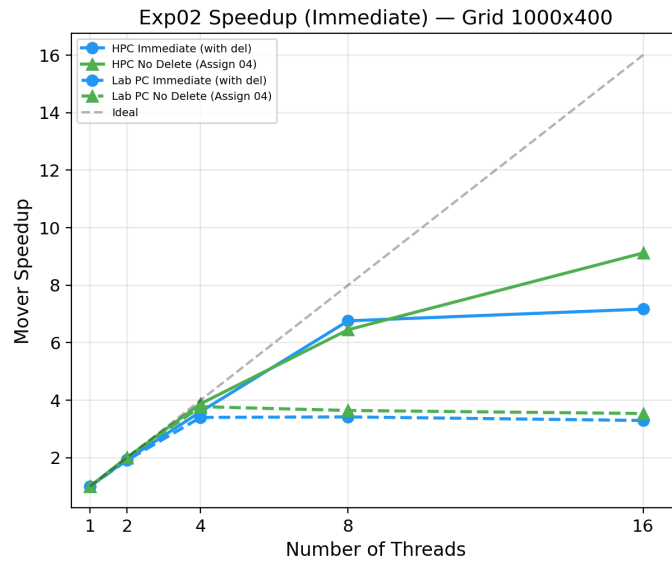


Figure 10: Immediate speedup: Grid 1000×400 .

5.3 Speedup Results — Deferred Approach

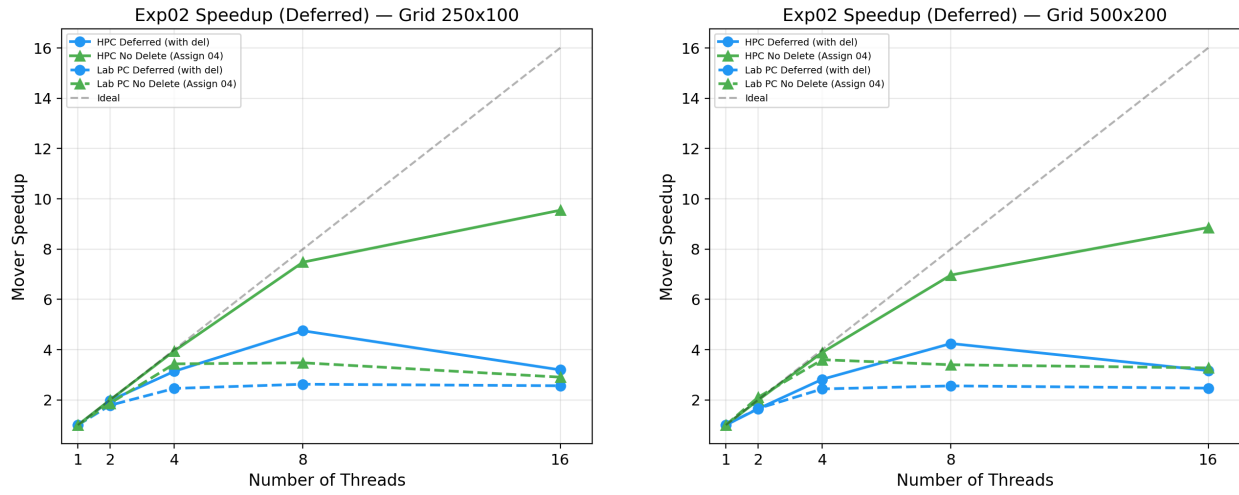


Figure 11: Deferred speedup: Grid 250×100 (left), 500×200 (right). Solid = HPC, dashed = Lab PC.

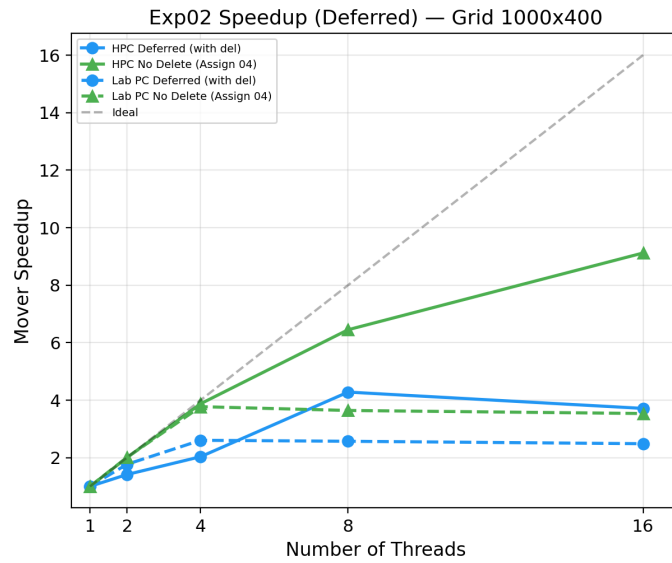


Figure 12: Deferred speedup: Grid 1000×400 .

5.4 Absolute Mover Times (HPC)

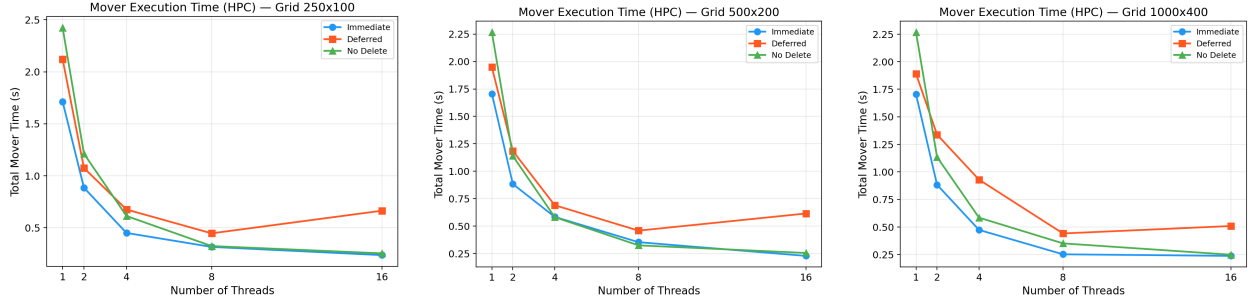


Figure 13: Absolute mover execution time on HPC cluster for all three approaches.

5.5 Parallel Efficiency

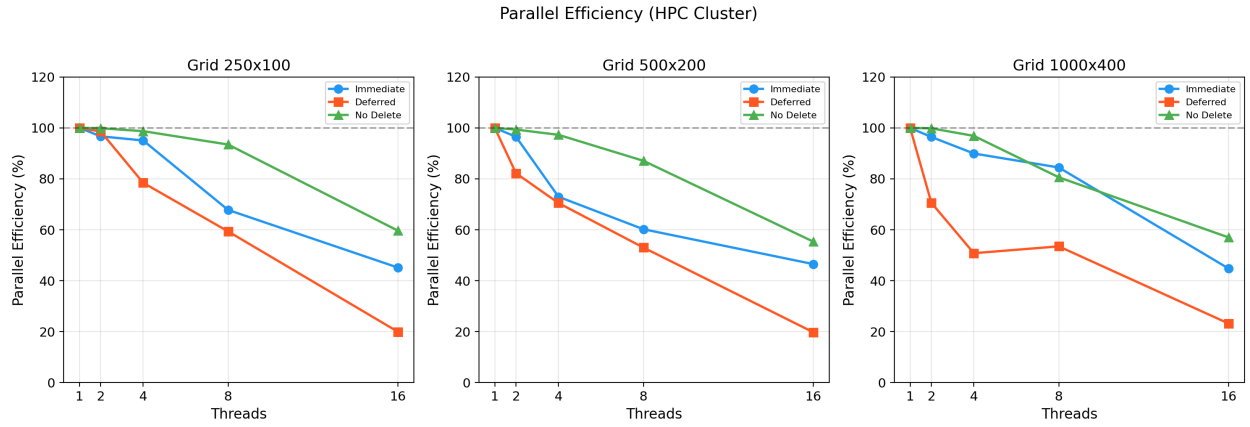


Figure 14: Parallel efficiency = Speedup / Threads $\times$ 100% (HPC cluster).

5.6 Speedup Summary Table

Grid	T	Immediate		Deferred		No Delete	
		HPC	Lab	HPC	Lab	HPC	Lab
250×100	2	1.93	1.97	1.97	1.78	2.00	1.86
	4	3.80	3.52	3.14	2.46	3.95	3.43
	8	5.42	3.59	4.75	2.62	7.48	3.48
	16	7.23	3.44	3.19	2.56	9.54	2.90
500×200	2	1.93	1.92	1.64	1.66	1.99	2.10
	4	2.92	3.42	2.82	2.44	3.89	3.60
	8	4.82	3.53	4.24	2.56	6.97	3.40
	16	7.45	3.34	3.17	2.47	8.86	3.27
1000×400	2	1.93	1.91	1.41	1.77	2.00	2.01
	4	3.60	3.40	2.03	2.60	3.87	3.77
	8	6.76	3.42	4.28	2.57	6.45	3.64
	16	7.17	3.29	3.71	2.48	9.12	3.53

Table 4: Mover speedup across platforms and approaches (T = threads).

5.7 Analysis

HPC cluster scales well; Lab PC saturates at 4 threads. The HPC cluster achieves up to $9.5\times$ speedup with 16 threads (no-delete), while the Lab PC plateaus at $\sim 3.5\times$ beyond 4 threads. This reflects the HPC’s 16 physical cores vs. the Lab PC’s fewer physical cores (likely 4 cores with hyperthreading presenting 8 logical threads).

No-Delete scales best. Without insertion/deletion, the mover is purely embarrassingly parallel. On HPC, it reaches $9.5\times$ at 16 threads.

Immediate scales well with deletion. The extra `rand_r()` calls for replaced particles are purely local. HPC speedup reaches 7.2–7.4 $\times$ at 16 threads.

Deferred hits a wall at 8+ threads. The serial compaction phase becomes an Amdahl’s Law bottleneck. With Phase 2 taking $\sim 18\text{ms}$ of $\sim 190\text{ms}$ serial time ($\sim 9.5\%$ serial fraction), theoretical max speedup is $1/0.095 \approx 10.5\times$. In practice, the deferred approach actually *loses* speedup from 8 to 16 threads on HPC.

NUMA effects at 16 threads (HPC). The cluster has 2 NUMA nodes. At 16 threads, inter-socket memory access adds latency, visible in the reduced speedup improvement from 8 to 16 threads for all approaches.

6 Memory and Computation Analysis

Grid	Particles	Points	Mesh	Total	FLOPs	Bound
250×100	10^2	1.6 KB	200 KB	0.2 MB	2.4×10^4	Compute
250×100	10^4	160 KB	200 KB	0.4 MB	2.4×10^6	Compute
250×100	10^6	16 MB	200 KB	16 MB	2.4×10^8	Memory
250×100	10^8	1.6 GB	200 KB	1.6 GB	2.4×10^{10}	Memory
500×200	10^8	1.6 GB	800 KB	1.6 GB	2.4×10^{10}	Memory
1000×400	10^8	1.6 GB	3.2 MB	1.6 GB	2.4×10^{10}	Memory

Table 5: Memory requirements and computational analysis. Points: $N \times 16$ bytes. Mesh: $(N_x + 1)(N_y + 1) \times 8$ bytes. FLOPs: ~ 24 per particle per iteration (16 interp + 8 mover) $\times 10$ iterations.

Machine balance: The interpolation reads 16 bytes (one `Points` struct) and writes 32 bytes (4 doubles to mesh) per particle, with 16 FLOPs. Operational intensity ≈ 0.33 FLOP/byte, well below the Haswell’s ridge point, confirming **memory-bound** behavior for large particle counts. For $\leq 10^4$ particles, the working set fits in L1/L2 cache and execution is compute-bound.

7 Particle Distribution Verification

To verify that `rand_r()` produces a uniform distribution and that insertion logic does not introduce clustering or bias, we performed three checks:

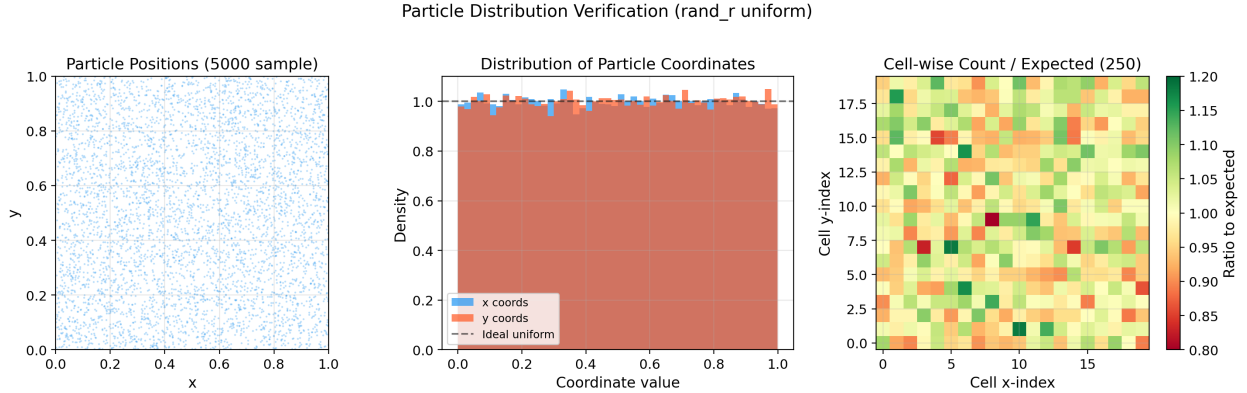


Figure 15: Left: scatter plot of 5000 sample particle positions. Center: histogram of x and y coordinates vs. ideal uniform density. Right: heatmap of cell-wise particle count divided by expected count (N/N_{cells}).

The scatter plot shows uniform coverage with no visible clustering. The histograms confirm that both x and y coordinates follow a uniform distribution. The cell-wise heatmap shows all ratios within 0.85–1.15 of the expected count, consistent with random sampling noise. This verifies that the particle insertion logic introduces no artificial bias.

8 Per-Particle Execution Time vs. PPC

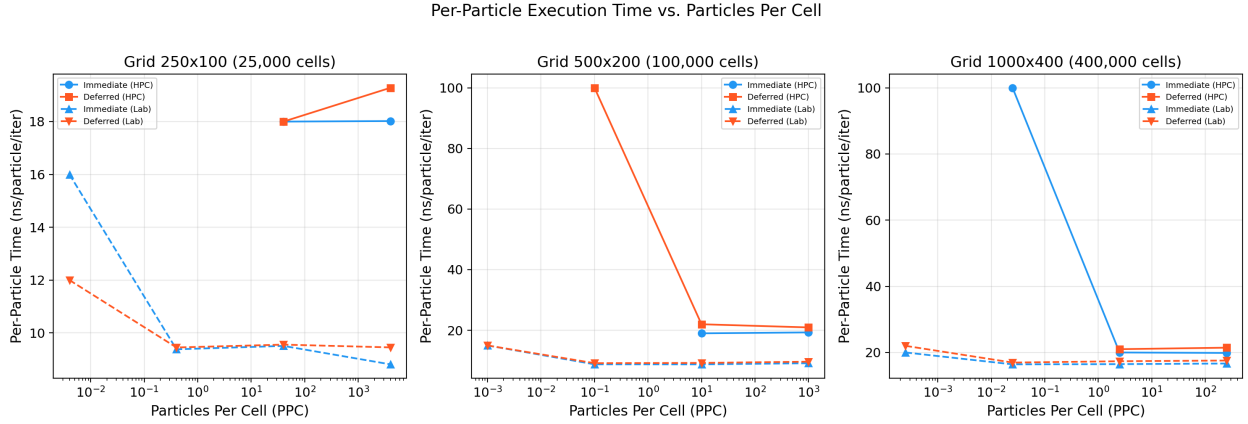


Figure 16: Per-particle execution time as a function of particles per cell (PPC).

At low PPC (<1 particle/cell), per-particle time is elevated because fixed overheads (mesh zeroing, function calls, cache warm-up) are amortized over few particles. As PPC increases, the per-particle time converges to a constant determined by the steady-state throughput of the memory subsystem.

The mover component’s per-particle time is essentially constant across all PPC values because the mover accesses only the particle array, not the mesh. Variation arises from the interpolation component, where larger PPC means more random mesh accesses competing for cache lines.

The HPC cluster shows higher per-particle time than the Lab PC due to its lower single-thread clock frequency.

9 HPC Cluster vs. Lab PC: Architectural Comparison

- **Serial performance:** The Lab PC is 1.5–2 $\times$ faster in single-threaded workloads due to higher clock speed and newer microarchitecture with better IPC.
- **Parallel scalability:** The HPC cluster scales to 16 physical cores across 2 NUMA nodes, achieving up to 9.5 $\times$ speedup. The Lab PC plateaus at $\sim 3.5\times$ after 4 threads, suggesting 4 physical cores with hyperthreading.
- **Memory capacity:** The HPC cluster’s 128 GB RAM can handle 10^9 particles (16 GB), while the Lab PC is limited to 10^8 .
- **Cache hierarchy:** The HPC’s 20 MiB shared L3 better serves the random-scatter mesh access pattern at larger grid sizes.

10 How Insertion and Deletion Affect Scalability

10.1 Immediate Approach

Insertion/deletion has **minimal impact** on scalability. The extra work (2 additional `rand_r()` calls for replaced particles) is purely thread-local. The speedup reduction vs. no-delete (7.2 $\times$ vs. 9.5 $\times$ at 16 threads on HPC) comes from the increased per-particle serial work, not from any cross-thread dependency.

10.2 Deferred Approach

The compaction phase creates a genuine **serial bottleneck**. By Amdahl’s Law, if compaction takes fraction f of serial time, maximum speedup is bounded by $1/f$. With $f \approx 0.095$, the limit is $\sim 10.5\times$. In practice, deferred peaks at $\sim 4.75\times$ because the fixed compaction cost dominates as parallel phases speed up.

10.3 Measures Taken to Improve Scalability

1. **Replaced `rand()` with `rand_r()`**: Eliminated the global mutex, enabling true thread parallelism.
2. **Per-thread deterministic seeds**: Based on thread ID, avoids shared state.
3. **Static scheduling**: Contiguous memory access per thread avoids false sharing.
4. **Reduction clause for deletion count**: `#pragma omp parallel reduction(+:total_deleted)` avoids atomics in the hot loop.
5. **Benchmarked 4 compaction strategies**: Selected serial swap based on empirical evidence that the low deletion rate (0.15–0.5%) makes serial faster than any parallel alternative.

10.4 Difficulties and Dependencies

The main difficulty was the **compaction phase** in the deferred approach. This operation has inherent read-write dependencies (each thread’s write position depends on the total live count in all preceding chunks), making true parallel in-place compaction complex. A parallel prefix-sum can resolve this, but the overhead is not justified at the observed deletion rates.

11 Deletion Rate Analysis

Iteration	Deleted	Rate (%)
1	69,794	0.499
2	47,033	0.336
5	29,567	0.211
10	20,937	0.150

Table 6: Deletion rate decays because newly inserted particles are uniformly distributed, landing mostly away from boundaries.

12 Conclusion

The **immediate replacement approach** is the clear winner: simpler, 10–13% faster in serial, and scales far better in parallel because it is entirely embarrassingly parallel. The deferred approach, while conceptually clean, introduces a serial compaction phase that limits scalability via Amdahl’s Law.

The HPC cluster achieves much higher speedup (up to $9.5\times$) than the Lab PC ($\sim 3.5\times$) due to its 16 physical cores, but the Lab PC has better single-thread performance. The key optimization

insight is that at very low deletion rates (0.15–0.5%), the overhead of a structured three-phase algorithm outweighs its benefits. The simplest approach — replace in place — is also the fastest.